AI/BIgData 인텐시브 과정

강사 장철원

Section

코스소개

Section 0

목차

❑ 크로스밸리데이션

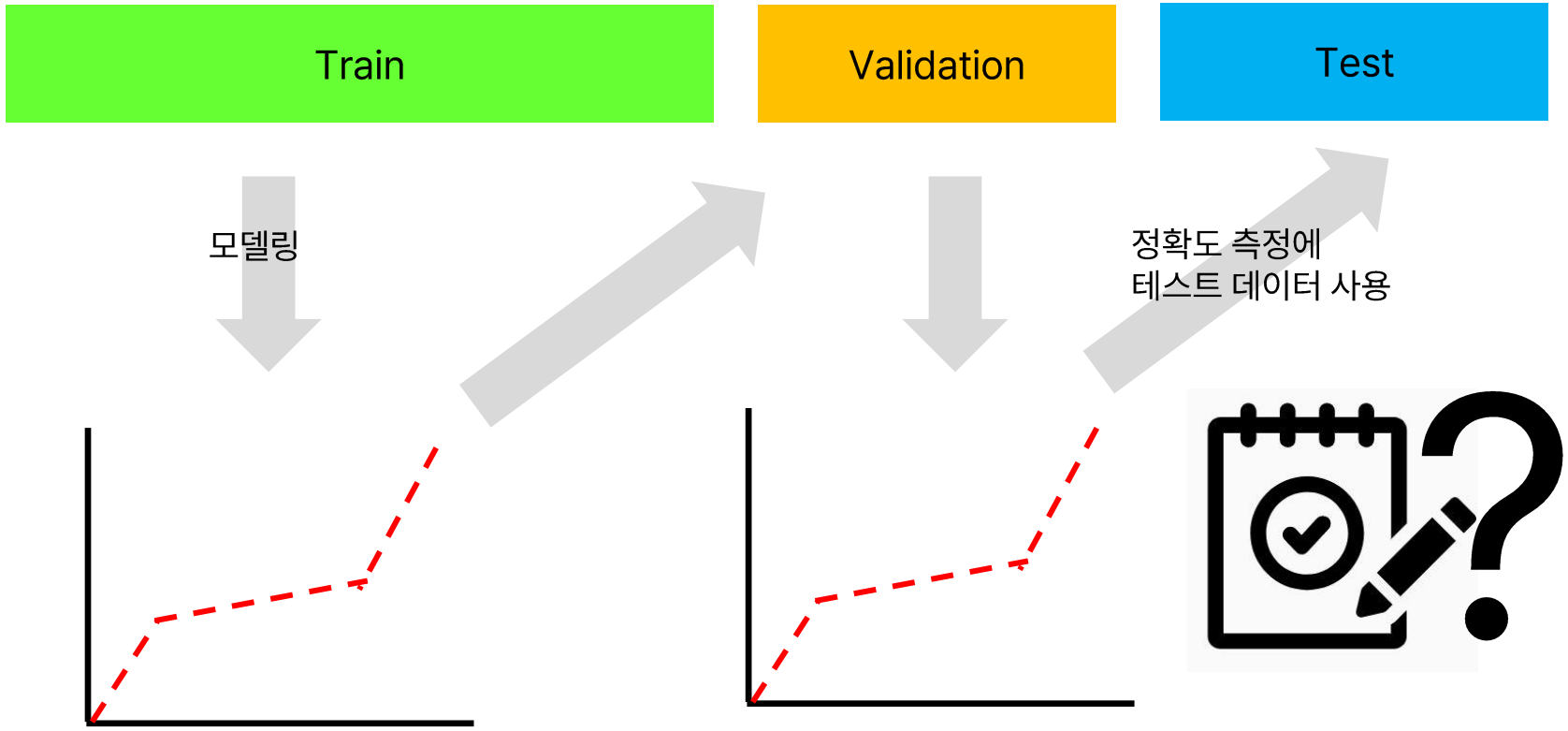
AI/BIgData 인텐시브 과정

Section 1. 크로스 밸리데이션

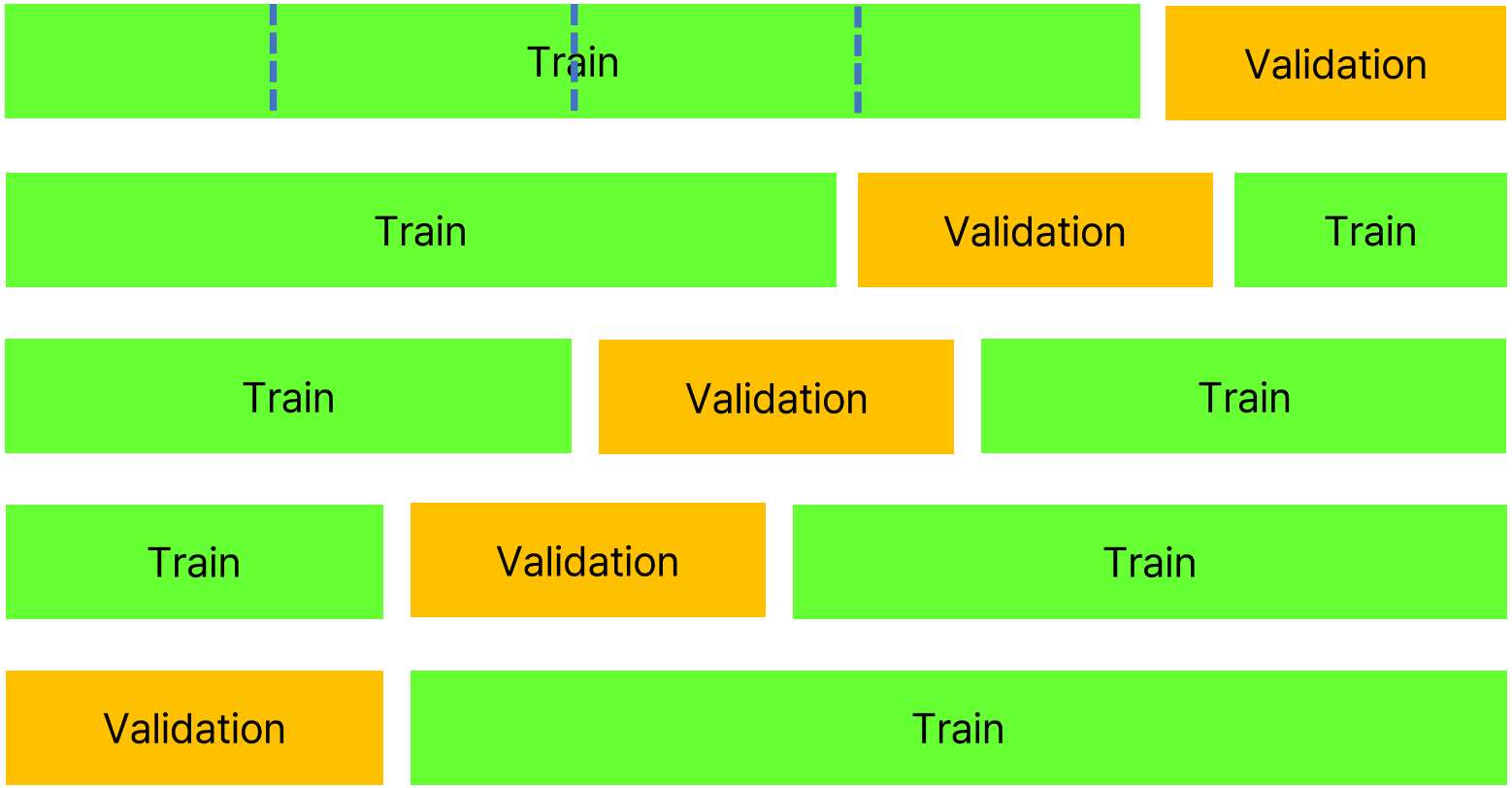
Section 1-1. 크로스 밸리데이션 분류

교차 검증(cross validation)

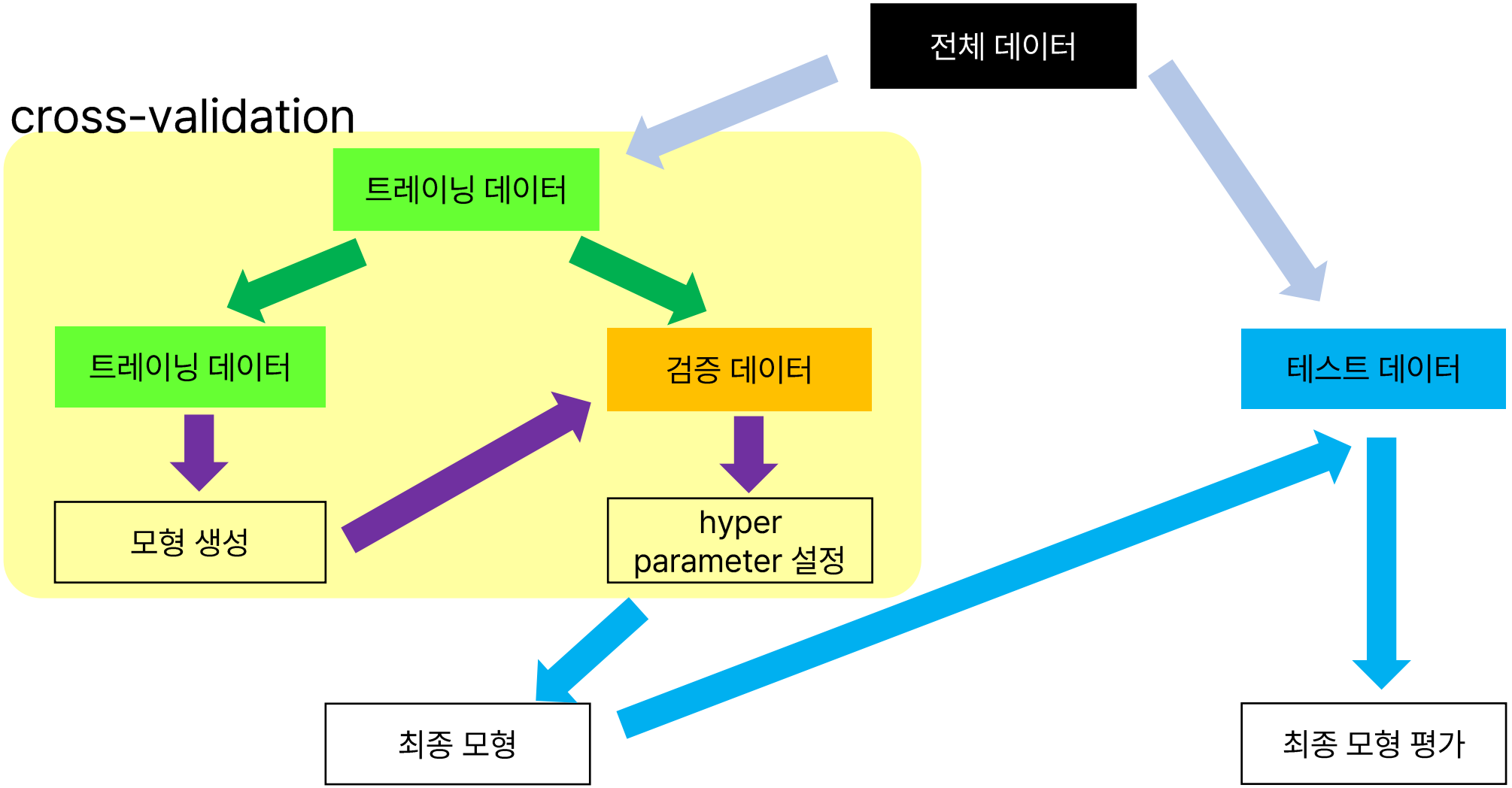
트레이닝 데이터 / 밸리데이션 데이터 / 테스트 데이터 분리



k fold cross validation



교차 검증(cross validation)



크로스 밸리데이션

In [1]:
```
import pandas as pd
df = pd.read_csv("./data/wine_data.csv")
```

In [2]:
```
features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',
            'Nonflavanoids', 'Proanthocyanins', 'Color','Hue', 'Dilution', 'Proline']

X = df[features]
y = df['class']
```

크로스 밸리데이션

In [3]:

```
# 트레이닝/테스트 데이터 분할
from sklearn.model_selection import train_test_split
X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

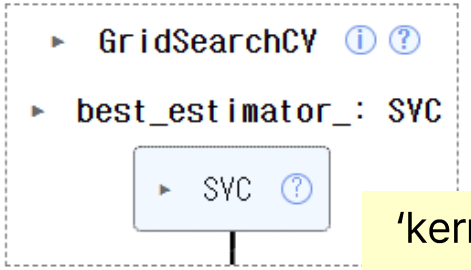
In [4]:

```
# 데이터 표준화
from sklearn.preprocessing import StandardScaler
std_scale = StandardScaler()
std_scale.fit(X_tn)
X_tn_std = std_scale.transform(X_tn)
X_te_std  = std_scale.transform(X_te)
```

크로스 밸리데이션

```
# 그리드 서치 학습
from sklearn.svm import SVC
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import GridSearchCV

param_grid= {'kernel': ('linear', 'rbf'),
             'C': [0.5, 1, 10, 100]}
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
svc = SVC(random_state=0)
grid_cv = GridSearchCV(svc, param_grid, cv=kfold, scoring='accuracy')
grid_cv.fit(X_tn_std, y_tn)
```

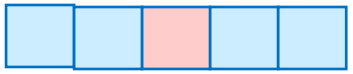


가능한 조합

'kernel' = 'linear' 'C' = 0.5	'kernel' = 'rbf' 'C' = 0.5	'kernel' = 'linear' 'C' = 1	'kernel' = 'rbf' 'C' = 1
'kernel' = 'linear' 'C' = 10	'kernel' = 'rbf' 'C' = 10	'kernel' = 'linear' 'C' = 100	'kernel' = 'rbf' 'C' = 100

실험 8번 필요

크로스 밸리데이션

	'kernel' = 'linear' 'C' = 0.5	'kernel' = 'rbf' 'C' = 0.5	'kernel' = 'linear' 'C' = 1	'kernel' = 'rbf' 'C' = 1	'kernel' = 'linear' 'C' = 10	'kernel' = 'rbf' 'C' = 10	'kernel' = 'linear' 'C' = 100	'kernel' = 'rbf' 'C' = 100
	0.88	0.96	0.88	0.92	0.88	0.92	0.88	0.92
	0.96	1.00	0.96	0.96	0.96	0.96	0.96	0.96
	0.92	0.96	0.92	0.96	0.92	0.96	0.92	0.96
	1.00	0.96	1.00	0.96	1.00	0.96	1.00	0.96
	0.84	1.00	0.84	1.00	0.84	1.00	0.84	1.00
평균	0.92	0.97	0.92	0.96	0.92	0.96	0.92	0.96

Section

크로스 밸리데이션 실습

In [6]:

```
# 그리드 서치 결과 확인
grid_cv.cv_results_
```

Out[6]:

```
{'mean_fit_time': array([0.00073509, 0.00103574, 0.00263205, 0.0008553 , 0.00104203,
        0.00121851, 0.00037541, 0.00083108]),
 'std_fit_time': array([0.00093357, 0.00087752, 0.00366261, 0.00104792, 0.00090734,
        0.00243702, 0.00046287, 0.00122318]),
 'mean_score_time': array([0.00141363, 0.00070448, 0.00013232, 0.00017943, 0.00028906,
        0.00015235, 0.00131979, 0.00119324]),
 'std_score_time': array([0.00237844, 0.00098571, 0.00026464, 0.00035887, 0.00043975,
        0.0003047 , 0.00136025, 0.00068336]),
 'param_C': masked_array(data=[0.5, 0.5, 1, 1, 10, 10, 100, 100],
              mask=[False, False, False, False, False, False, False, False],
        fill_value='?',
             dtype=object),
 'param_kernel': masked_array(data=['linear', 'rbf', 'linear', 'rbf', 'linear', 'rbf',
                    'linear', 'rbf'],
              mask=[False, False, False, False, False, False, False, False],
        fill_value='?',
             dtype=object),
 'params': [{'C': 0.5, 'kernel': 'linear'},
  {'C': 0.5, 'kernel': 'rbf'},
  {'C': 1, 'kernel': 'linear'},
  {'C': 1, 'kernel': 'rbf'},
  {'C': 10, 'kernel': 'linear'},
  {'C': 10, 'kernel': 'rbf'},
  {'C': 100, 'kernel': 'linear'},
  {'C': 100, 'kernel': 'rbf'}],
 'split0_test_score': array([0.88888889, 0.96296296, 0.88888889, 0.92592593, 0.88888889,
        0.92592593, 0.88888889, 0.92592593]),
 'split1_test_score': array([0.96296296, 1.        , 0.96296296, 0.96296296, 0.96296296,
        0.96296296, 0.96296296, 0.96296296]),
```

Section

크로스 밸리데이션 실습

In [7]:

```
# 그리드 서치 결과 확인(데이터프레임)

import numpy as np
import pandas as pd

np.transpose(pd.DataFrame(grid_cv.cv_results_))
```

	0	1	2	3	4	5	6	7
mean_fit_time	0.000735	0.001036	0.002632	0.000855	0.001042	0.001219	0.000375	0.000831
std_fit_time	0.000934	0.000878	0.003663	0.001048	0.000907	0.002437	0.000463	0.001223
mean_score_time	0.001414	0.000704	0.000132	0.000179	0.000289	0.000152	0.00132	0.001193
std_score_time	0.002378	0.000986	0.000265	0.000359	0.00044	0.000305	0.00136	0.000683
param_C	0.5	0.5	1	1	10	10	100	100
param_kernel	linear	rbf	linear	rbf	linear	rbf	linear	rbf
params	{'C': 0.5, 'kernel': 'linear'}	{'C': 0.5, 'kernel': 'rbf'}	{'C': 1, 'kernel': 'linear'}	{'C': 1, 'kernel': 'rbf'}	{'C': 10, 'kernel': 'linear'}	{'C': 10, 'kernel': 'rbf'}	{'C': 100, 'kernel': 'linear'}	{'C': 100, 'kernel': 'rbf'}
split0_test_score	0.888889	0.962963	0.888889	0.925926	0.888889	0.925926	0.888889	0.925926
split1_test_score	0.962963	1.0	0.962963	0.962963	0.962963	0.962963	0.962963	0.962963
split2_test_score	0.925926	0.962963	0.925926	0.962963	0.925926	0.962963	0.925926	0.962963
split3_test_score	1.0	0.961538	1.0	0.961538	1.0	0.961538	1.0	0.961538
split4_test_score	0.846154	1.0	0.846154	1.0	0.846154	1.0	0.846154	1.0
mean_test_score	0.924786	0.977493	0.924786	0.962678	0.924786	0.962678	0.924786	0.962678
std_test_score	0.054014	0.018384	0.054014	0.023431	0.054014	0.023431	0.054014	0.023431
rank_test_score	5	1	5	2	5	2	5	2

크로스 밸리데이션

In [8]:
```
# 베스트 스코어
grid_cv.best_score_
```

Out[8]:
```
0.9774928774928775
```

In [9]:
```
# 베스트 하이퍼파라미터
grid_cv.best_params_
```

Out[9]:
```
{'C': 0.5, 'kernel': 'rbf'}
```

In [10]:
```
# 최종 모형
clf = grid_cv.best_estimator_
print(clf)
```

```
SVC(C=0.5, random_state=0)
```

크로스 밸리데이션

In [11]:
```
# 크로스 밸리데이션 스코어 확인(1)
from sklearn.model_selection import cross_validate
metrics = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
cv_scores = cross_validate(clf, X_tn_std, y_tn,
                           cv=kfold, scoring=metrics)
cv_scores
```

Out[11]:
```
{'fit_time': array([0.        , 0.00316906, 0.        , 0.00152636, 0.00160766]),
 'score_time': array([0.00351453, 0.00155187, 0.00566816, 0.00306392, 0.0028975 ]),
 'test_accuracy': array([0.96296296, 1.        , 0.96296296, 0.96153846, 1.        ]),
 'test_precision_macro': array([0.96296296, 1.        , 0.96969697, 0.96969697, 1.        ]),
 'test_recall_macro': array([0.96666667, 1.        , 0.96296296, 0.95833333, 1.        ]),
 'test_f1_macro': array([0.9628483 , 1.        , 0.96451914, 0.96190476, 1.        ])}
```

In [12]:
```
# 크로스 밸리데이션 스코어 확인(2)
from sklearn.model_selection import cross_val_score
cv_score = cross_val_score(clf, X_tn_std, y_tn,
                           cv=kfold, scoring='accuracy')
print(cv_score)
print(cv_score.mean())
print(cv_score.std())
```

```
[0.96296296 1.         0.96296296 0.96153846 1.        ]
0.9774928774928775
0.01838434849561446
```

크로스 밸리데이션

In [13]:

```
# 예측
pred_svm = clf.predict(X_te_std)
print(pred_svm)
```

```
[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 1 0 1 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```

In [14]:

```
# 정확도
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_te, pred_svm)
print(accuracy)
```

```
1.0
```

크로스 밸리데이션

In [15]:

```
# confusion matrix 확인
from sklearn.metrics import confusion_matrix
conf_matrix = confusion_matrix(y_te, pred_svm)
print(conf_matrix)
```

```
[[16  0  0]
 [ 0 21  0]
 [ 0  0  8]]
```

In [16]:

```
# 분류 레포트 확인
from sklearn.metrics import classification_report
class_report = classification_report(y_te, pred_svm)
print(class_report)
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        16
           1       1.00      1.00      1.00        21
           2       1.00      1.00      1.00         8

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

AI/BIgData 인텐시브 과정

Section 1. 크로스 밸리데이션

Section 1-1. 크로스 밸리데이션 예측

크로스 밸리데이션

In [2]:
```
import pandas as pd
df = pd.read_csv("./data/house_prices.csv")
```

In [3]:
```
features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']

X = df[features]
y = df['MEDV']
```

In [4]:
```
# 트레이닝/테스트 데이터 분할
from sklearn.model_selection import train_test_split
X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]:
```
# 데이터 표준화
from sklearn.preprocessing import StandardScaler
std_scale = StandardScaler()
std_scale.fit(X_tn)
X_tn_std = std_scale.transform(X_tn)
X_te_std  = std_scale.transform(X_te)
```

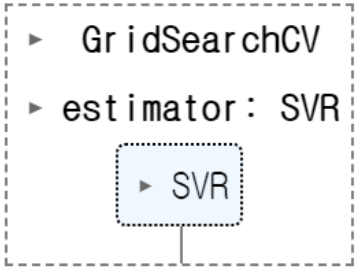
크로스 밸리데이션

In [10]:

```
# 그리드 서치 학습
from sklearn import svm
from sklearn.model_selection import KFold
from sklearn.model_selection import GridSearchCV

param_grid= {'kernel': ('linear', 'rbf'),
             'C': [0.5, 1, 10, 100]}
kfold = KFold(n_splits=5, shuffle=True, random_state=0)
svr = svm.SVR()
grid_cv = GridSearchCV(svr, param_grid, cv=kfold, scoring='neg_mean_squared_error')
grid_cv.fit(X_tn_std, y_tn)
```

Out[10]:



Section

크로스 밸리데이션 실습

In [11]:

```
# 그리드 서치 결과 확인(데이터프레임)

import numpy as np
import pandas as pd

np.transpose(pd.DataFrame(grid_cv.cv_results_))
```

Out[11]:

	0	1	2	3	4	5	6	7
mean_fit_time	0.004308	0.003131	0.004675	0.00253	0.015536	0.005196	0.096247	0.013163
std_fit_time	0.000869	0.001588	0.001412	0.001406	0.003971	0.004069	0.038219	0.000805
mean_score_time	0.000587	0.001271	0.000398	0.001447	0.000823	0.000783	0.00058	0.001466
std_score_time	0.000484	0.000721	0.000488	0.001005	0.000702	0.000744	0.000539	0.000526
param_C	0.5	0.5	1	1	10	10	100	100
param_kernel	linear	rbf	linear	rbf	linear	rbf	linear	rbf
params	{'C': 0.5, 'kernel': 'linear'}	{'C': 0.5, 'kernel': 'rbf'}	{'C': 1, 'kernel': 'linear'}	{'C': 1, 'kernel': 'rbf'}	{'C': 10, 'kernel': 'linear'}	{'C': 10, 'kernel': 'rbf'}	{'C': 100, 'kernel': 'linear'}	{'C': 100, 'kernel': 'rbf'}
split0_test_score	-17.445032	-36.17043	-17.386875	-27.228367	-17.458346	-17.409092	-17.446742	-12.562522
split1_test_score	-18.593747	-16.916761	-18.521081	-11.816775	-18.665759	-5.996846	-18.667153	-9.235812
split2_test_score	-40.571874	-76.712963	-38.982199	-57.052392	-37.459318	-26.609204	-37.063074	-15.662599
split3_test_score	-23.262262	-34.792208	-23.221267	-24.185331	-23.007632	-9.182264	-22.978231	-6.061032
split4_test_score	-17.088043	-33.352135	-17.18929	-23.301751	-17.192183	-9.34963	-17.19154	-7.405381
mean_test_score	-23.392191	-39.588899	-23.060142	-28.716923	-22.756648	-13.709407	-22.669348	-10.185469
std_test_score	8.868985	19.82833	8.255832	15.103569	7.642017	7.470993	7.490986	3.501173
rank_test_score	6	8	5	7	4	2	3	1

크로스 밸리데이션

In [12]:
```
# 베스트 스코어
grid_cv.best_score_
```

Out[12]:
```
-10.185469344281149
```

In [13]:
```
# 베스트 하이퍼파라미터
grid_cv.best_params_
```

Out[13]:
```
{'C': 100, 'kernel': 'rbf'}
```

In [14]:
```
# 최종 모형
model = grid_cv.best_estimator_
print(model)
```

```
SVR(C=100)
```

In [15]:
```
# 예측
pred_svm = model.predict(X_te_std)
print(pred_svm)
```

```
[22.36813301 22.68383242 24.87514322 10.71088583 20.84027476 19.73537584
 23.616982   19.95204534 20.95058207 22.88723384 18.0129821  10.34899842
 14.71816571  5.72642875 47.13543116 31.19225615 23.92516254 36.38402412
 29.8542525  21.3802061  23.4583459  19.83747957 20.28994301 27.14446529
 20.76220706 21.44860078 16.90027694 17.73675701 37.75599541 18.56694782
```

크로스 밸리데이션

In [16]:

```
1 # 모형 평가-MSE
2 from sklearn.metrics import mean_squared_error
3 mse = mean_squared_error(y_te, pred_svm)
4 print(mse)
```

```
17.51711113685591
```

감사합니다.

Q & A